

Egy egyszerű és hatékony algoritmus egy minimális feszítőfa helyettesítő éleinek megtalálására

David A. Bader, Paul Burkhardt

<http://jgaa.info/> vol. 26, no. 4, pp. 577-588 (2022)

Tóth Ábel Tibor, Nagy Bence

2025. november 17.

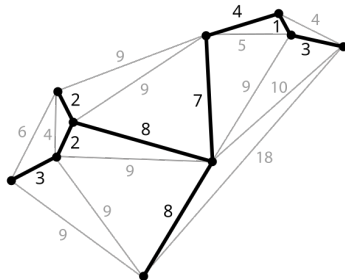
Áttekintés: minimális feszítőfa, csereél keresése

Definíció:

Legyen $G = (V, E)$ egy irányítatlan, súlyozott gráf, ahol $n = |V|$ és $m = |E|$, és minden $e \in E$ élhez tartozik egy $w(e)$ súly. Egy minimális feszítőfa $T = MST(G)$ a gráf olyan $n - 1$ élből álló részhalmaza, amely összeköti az összes csúcsot, és az élek összsúlya minimális.

Definíció:

A csereél az a legkisebb súlyú él, amely újra összeköti a minimális feszítőfát (egy minimális feszítőfabeli él törlése után).



Kép forrás: Wikipédia

Az algoritmus célja és eredménye (1.)

Cél:

Keressük meg a minimális költségű csereélét minden minimális feszítőfabeli élhez.

- Az algoritmus képes megtalálni ezeket, ha a nem min feszfabeli élek élsúly szerint növekvő sorrendben rendezve megvannak határozva:
 - **Idő- és tárbonyolultság:** $\mathcal{O}(m + n)$
(n : csúcsok száma, m : élek száma)
- A cikk előtt lehetett tudni, hogy ez elméletileg lehetséges, de ez az első algoritmikus megoldás erre.

Az algoritmus célja és eredménye (2.)

- Megjegyzés: a nem min.feszfabeli éleket rendezve kapjuk meg, ha az MST-t a Kruskal algoritmussal keressük meg.
 - Ha nem a Kruskal: a nem MST éleket $\mathcal{O}(n)$ időbonyolultsággal rendezhetjük, ha ismerünk egy felső korlátot a lehetséges értékekre.
- A cikk előtti legjobb eredmény egy $\mathcal{O}(m\alpha(m,n))$ időbonyolultságú algoritmus volt Tarjan-tól 1979-ből. (megj.: $\alpha(m,n)$: Inverse Ackermann's function)

Definíció:

A diszjunkt halmaz adatstruktúra diszjunkt halmazok gyűjteményét tartja fenn: $\mathcal{S} = \{S_1, S_2, \dots, S_k\}$. Minden halmazt egy **képviselőjével** azonosítjuk, amely tagja a halmaznak.

Az algoritmus a következő műveleteket használja a diszjunkt halmaz adatstruktúrán:

- **makeSet(v)**: létrehoz egy diszjunkt halmaz-t és elhelyezi benne v élet.
- **find(v)**: megkeresi azt a diszjunkt halmazt, amelyben v él van és visszaadja annak képviselőjét.
- **link**: 2 diszjunkt halmazt össze unióz.

Gabow-Tarjan algoritmusa a diszjunkt halmazok egy speciális esetére

- Gabow és Tarjan létrehoztak egy speciális algoritmus a diszjunkt halmazok egy speciális esetére.
- Speciális eset: az összes lehetséges uniós szerekezetet (azaz a Union-Tree-t) ismert legyen.
- Esetünkben az eredeti MST lesz a Union-Tree.
- Az algoritmus **m darab unió és find műveletet** hajt végre, **n darab elemen**: $\mathcal{O}(m + n)$ futási időben, $\mathcal{O}(n)$ tárhelykomplexitással

Algoritmus bemutató

```
1: procedure PATHLABEL( $s, t, e$ )
2: if  $IN[s] < IN[t] < OUT[s]$  then                                ▷  $s$  is ancestor of  $t$ 
3:   return
4: if  $IN[t] < IN[s] < OUT[t]$  then                                ▷  $t$  is ancestor of  $s$ 
5:    $PLAN \leftarrow ANCESTOR$ ;  $k_1 \leftarrow IN[t]$ ;  $k_2 \leftarrow IN[s]$ 
6: else
7:   if  $IN[s] < IN[t]$  then
8:      $PLAN \leftarrow LEFT$ ;  $k_1 \leftarrow OUT[s]$ ;  $k_2 \leftarrow IN[t]$       ▷  $s$  is left of  $t$ 
9:   else
10:     $PLAN \leftarrow RIGHT$ ;  $k_1 \leftarrow OUT[t]$ ;  $k_2 \leftarrow IN[s]$     ▷  $s$  is right of  $t$ 
11:   end if
12: end if
13:  $v \leftarrow s$ 
14: while  $k_1 < k_2$  do                                           ▷ Detecting when below LCA( $s, t$ )
15:   if  $FIND(v) = v$  then                                         ▷ If true, set replacement edge for ( $v, P[v]$ )
16:      $R(v, P[v]) \leftarrow e$                                      ▷ Set the replacement edge
17:      $LINK(v)$                                                   ▷ Union the disjoint sets of  $v$  and  $P[v]$ 
18:   end if
19:    $v \leftarrow FIND(v)$ 
20:   switch  $PLAN$  do
21:     case  $ANCESTOR$ :  $k_2 \leftarrow IN[v]$ 
22:     case  $LEFT$ :  $k_1 \leftarrow OUT[v]$ 
23:     case  $RIGHT$ :  $k_2 \leftarrow IN[v]$ 
24:   end while
```

Algoritmus bemutatása

- 1: Root the MST T at arbitrary vertex v_r and store parents in P .
- 2: $P[v_r] \leftarrow v_r$ ▷ root's parent points to root
- 3: Run DFS on T , setting $IN[v]$ and $OUT[v]$ to the counter value when v is first and last visited.
- 4: **for all** vertices $v \in V$ **do**
- 5: $\text{MAKESET}(v)$ ▷ Initialize the disjoint sets
- 6: **end for**
- 7: **for all** edges $e \in E_T$ **do**
- 8: $R_e \leftarrow \emptyset$ ▷ Initialize the replacement edges
- 9: **end for**
- 10: **for** $k \leftarrow 1$ **to** $m - n + 1$ **do**
- 11: $(v_i, v_j) \leftarrow e_k$ ▷ Scan the $m - n + 1$ sorted non-MST edges
- 12: $\text{PATHLABEL}(v_i, v_j, (v_i, v_j))$
- 13: $\text{PATHLABEL}(v_j, v_i, (v_i, v_j))$
- 14: **end for**

Legfontosabb él (Most Vital Edge)

- **Definíció:** Az az él, amelynek eltávolítása a legnagyobb növekedést okozza az MST (minimális feszítőfa) súlyában.
- **Megjegyzés:** Ha G -ben van híd, a legfontosabb él nincs értelmezve.
- **Kulcsállítás:** A legfontosabb él az MST-ben van.
- **Az algoritmus hozzájárulása:**
 - A csereélek meghatározása után a legfontosabb él kiválasztható $\mathcal{O}(n)$ időben. Csak meg kell keresni, hogy melyik MST-beli él súlyának a különbsége legnagyobb a csereélének súlyával.
 - Így: a legfontosabb él kiszámítható $\mathcal{O}(m + n)$ időben, ha adott az eredeti MST és a nem MST élek rendezve.